Rockchip Developer Guide Dual Storage

ID:RK-KF-YF-461

Release Version: V1.5.1

Release Date: 2025-01-06

Security Level: □Top-Secret □Secret □Internal ■Public

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document introduces the support principles and corresponding configurations of the Rockchip SDK for dual storage solutions.

Product Version

Chipset	Kernel Version
Chips with multiple IO-independent storage	kernel 4.19 and above

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision Records

Version	Author	Date	Change Description
V1.0.0	Lin Dingqiang	2022-07-12	Initial version
V1.1.0	Zhao Yifeng	2022-07-22	Added vendor stroage support
V1.2.0	Lin Dingqiang	2023-06-25	Added RK PCIe EP dual storage solution support, added firmware packaging and OTA upgrade instructions
V1.3.0	Lin Dingqiang	2023-08-08	Added Nor Flash firmware partition layout trimming suggestions
V1.4.0	Lin Dingqiang	2023-10-20	Added pcie_idb.img image creation instructions
V1.5.0	Lin Dingqiang	2024-08-23	Added RK PCIe EP dual storage solution firmware packaging SPI Nand solution
V1.5.1	Lin Dingqiang	2025-01-06	Corrected layout errors

Contents

1. Introduction

Multi-Storage Support

RK SOC typically supports multiple IO-independent storage controllers. For instance, RK3568 supports both Nor flash and eMMC, while RK3588 supports Nor flash and PCIe SSD, with each storage controller's IO being mutually exclusive. Storage devices can be categorized into Bootable and Non-bootable types based on whether they can be detected by the mask code, such as:

- Bootable devices: eMMC, SPI Flash, PP Nand
- Non-bootable devices: NVMe SSD, SATA (only Non-bootable storage scheme mask code cannot boot normally)

Application Solutions

To support Non-bootable main memory solutions, it is necessary to pair with a Bootable storage device. For example:

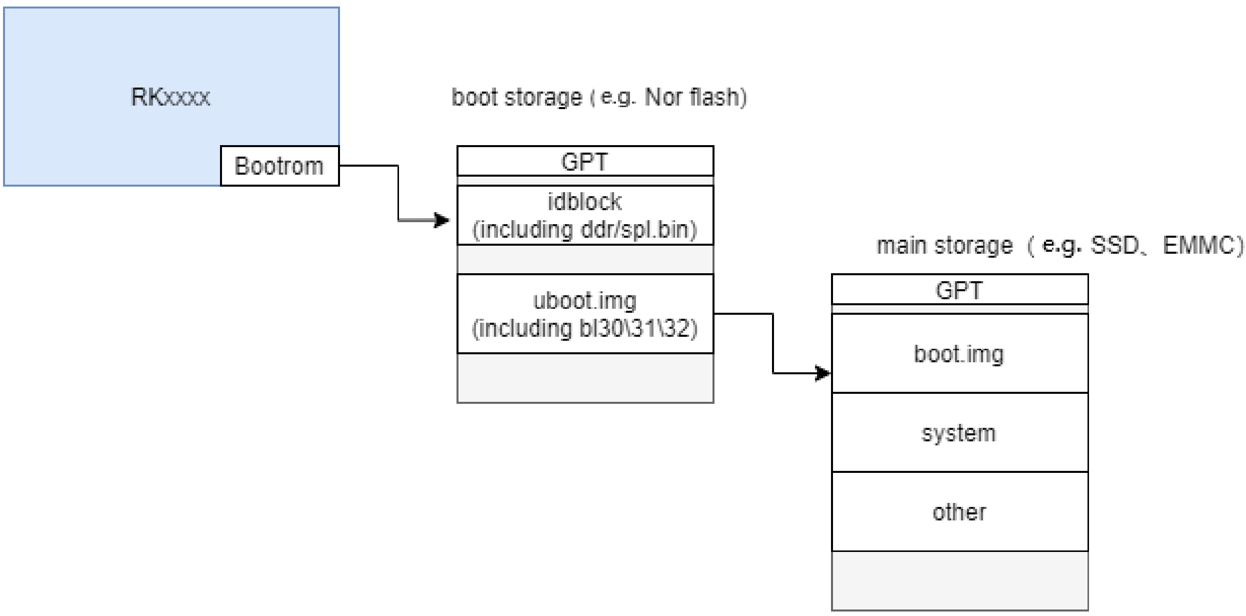
- Nor + NVMe SSD/SATA to achieve large-capacity storage support

In addition to this, there are other applications:

- Nor + eMMC to implement a special firmware protection mechanism, such as recoverable small system storage in Nor to prevent irreparable anomalies in eMMC.

2. Principle

2.1 RK General Dual Storage Boot Scheme



RK Dual Storage Scheme Process and Firmware Information:

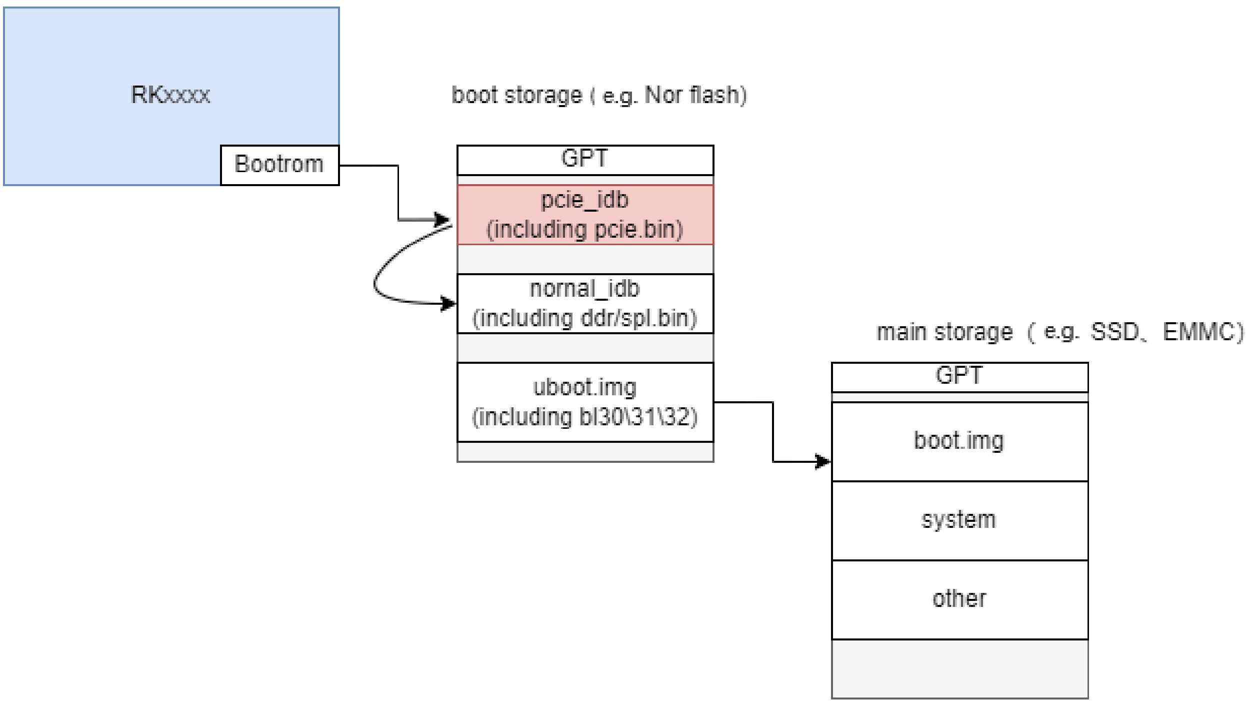
Boot Stage	Firmware Storage Location	Primary Storage Initialized in This Stage	Required Configuration
Bootrom	Chip Mask ROM	bootable storage, such as Nor	\
SPL	bootable storage, such as Nor	bootable storage, such as Nor	\
u-boot	bootable storage, such as Nor	main storage, such as NVMe	u-boot special config
kernel	main storage, such as NVMe	main storage, such as NVMe	Kernel standard config

Explanation:

- During the u-boot stage, the default initialization is for the storage type used by the previous level, so to switch to initializing the primary storage, such as NVMe, u-boot special configuration is required. For details, refer to the "Configuration" section.

2.2 RK PCIe EP Dual Storage Solution Boot

RK SOCs support the PCIe EP function, which allows for the design of PCIe EP standard cards to interface with general-purpose PCs. Therefore, there is a need for an early initialization of the PCIe controller, hence the addition of the pcie_idb.img image to achieve this function. The process is as follows:



RK Dual Storage Solution Process and Firmware Information:

Boot Stage	Firmware Storage Location	Primary Storage Initialized at This Stage	Required Configuration
Bootrom	Chip Mask ROM	bootable storage, such as Nor	\
PCIe.bin	bootable storage, such as Nor	NULL	\
SPL	bootable storage, such as Nor	bootable storage, such as Nor	\
u-boot	bootable storage, such as Nor	main storage, such as NVMe	u-boot special config
kernel	main storage, such as NVMe	main storage, such as NVMe	Kernel standard config

Explanation:

- During the u-boot stage, the storage type used by the previous level is initialized by default, so to switch to initializing the primary storage, such as NVMe, u-boot special configuration needs to be added. For detailed reference, see the "Configuration" section.

3. Configuration

As described in the "Principle" section, the RK dual-storage solution SDK modifies the initialization of the target storage during the u-boot phase. Therefore, the u-boot configuration should focus on the following main storage driver configuration, main storage transition configuration, and the use of Embedded Kernel DTB configuration.

3.1 U-Boot Configuration

3.1.1 Main Storage Driver Configuration

Refer to the document "Rockchip_Developer_Guide_UBoot_Nextdev_CN.pdf", section "CH05-Driver Modules":

- For eMMC, refer to the "Storage" section; it is usually default compatible.
- For PCIe NVMe, refer to the "PCIe" section.

3.1.2 Main Storage Transition Configuration

defconfig adds configuration;

```
CONFIG_ROCKCHIP_BOOTDEV="nvme 0"     # Optional "nvme"-NVMe "mmc"-eMMC "scsi"-SATA
CONFIG_ROCKCHIP_EMMC_IOMUX=y         # Nor + eMMC scheme completes eMMC iomux in spl.bin
```

3.1.3 NVMe/SATA Configuration with Embedded Kernel DTB

RK u-boot is divided into two stages:

- Early u-boot initialization, using the standard u-boot dtb with minimal information, refer to the "Storage Driver Configuration" section for related configurations.
- Later stage of u-boot, optional:
  - using kernel dtb: Utilize the kernel dtb file from the subsequent image, parse and initialize the corresponding device drivers during the boot process.
  - using embedded dtb: Package the dtb files from a specific directory of u-boot into the u-boot image, parse and initialize the corresponding device drivers during the boot process —— Use this configuration.

using embedded dtb

defconfig configuration:

```
CONFIG_EMBED_KERNEL_DTB_ALWAYS=y
CONFIG_EMBED_KERNEL_DTB_PATH="dts/rk3588-evb1-lp4-v10.dtb"     # Directory where dtb is
stored
CONFIG_SPL_FIT_IMAGE_KB=2560                                   # Typically, the u-boot
firmware will increase in size when using embedded dtb
```

Adding dtb files:

- Compile the kernel dtb required for the kernel and store it in the u-boot/dts/ directory, it is recommended to continue using the kernel dtb naming for differentiation.
- If updates are needed later, simply replace the dtb.

3.2 Kernel Configuration

3.3 Android Configuration

Modify the storage medium to the corresponding pcie in Android for partition mounting.

```
@sys2_206:~/4_Android12_29_sdk/device/Rockchip/rk3588$ git diff
--- a/BoardConfig.mk
+++ b/BoardConfig.mk
@@ -25,7 +25,7 @@ PRODUCT_KERNEL_CONFIG ?= rockchip_defconfig pcie_wifi.config

 #BOARD_AVB_ENABLE := true
 # used for fstab_generator, sdmmc controller address
-PRODUCT_BOOT_DEVICE := fe2e0000.mmc
+PRODUCT_BOOT_DEVICE := fe180000.pcie
 PRODUCT_SDMMC_DEVICE := fe2c0000.mmc

 SF_PRIMARY_DISPLAY_ORIENTATION := 0
```

4. Firmware Partition Layout and Packaging

4.1 RK General Dual-Storage Firmware Solution

The default SDK only supports single-storage firmware packaging, customers can refer to the following scripts to modify the SDK packaging scripts, or directly use the new packaging scheme to achieve dual-storage firmware packaging.

Tool Directory

```
.
├── afptool
├── gen-package-file.sh
├── Image                          # Main storage firmware
│   ├── boot.img
│   ├── MiniLoaderAll.bin
│   ├── parameter.txt
│   └── userdata.img
├── Image-nor                      # Boot storage firmware
│   ├── MiniLoaderAll.bin
│   ├── parameter.txt
│   └── uboot.img
├── mkupdate.sh
└── rkImageMaker
```

New Image-nor Directory

The compiled Nor flash firmware is placed in the Image-nor directory, and the layout of parameter.txt can be referenced as follows:

```
CMDLINE:mtdparts=rk29xxnand:0x00000080@0x00000000(gpt),0x00000800@0x00000080(idb),0x00000800
@0x00001000(vnvm),0x00002000@0x00001800(uboot)
```

Modify mkupdate.sh Script to Support Dual-Storage Packaging

```bash
#!/bin/bash
# Author: kenjc@rock-chips.com
# 2021-08-13
# Use: ./mkupdate.sh PLATFORM IMAGE_PATH to pack update.img

declare -A vendor_id_map
vendor_id_map["rk3588"]="-RK3588"
vendor_id_map["rk356x"]="-RK3568"
vendor_id_map["rk3326"]="-RK3326"
vendor_id_map["px30"]="-RKPX30"
vendor_id_map["rk3368"]="-RK330A"
vendor_id_map["rk322x"]="-RK322A"
vendor_id_map["rk3399pro"]="-RK330C"
vendor_id_map["rk3328"]="-RK322H"
vendor_id_map["rk3288"]="-RK32"
vendor_id_map["rk3126c"]="-RK312A"
vendor_id_map["rk3399"]="-RK330C"

readonly PLATFORM=$1
readonly IMAGE_PATH=$2
readonly IMAGE_PATH_NOR=$3
readonly PACKAGE_FILE=package-file-tmp

echo "packing update.img with $IMAGE_PATH ${vendor_id_map[$PLATFORM]}"

pause() {
  echo "Press any key to quit:"
  read -n1 -s key
  exit 1
}


if [ ! -f "$IMAGE_PATH/parameter.txt" ]; then
    echo "Error:No found parameter!"
#   pause
fi

echo "regenernate $PACKAGE_FILE..."
if [ -f "$PACKAGE_FILE" ]; then
    rm -rf $PACKAGE_FILE
fi
./gen-package-file.sh $IMAGE_PATH > $PACKAGE_FILE

echo "start to make emmc update.img..."
./afptool -pack ./ $IMAGE_PATH/update.img $PACKAGE_FILE || pause
./rkImageMaker ${vendor_id_map[$PLATFORM]} $IMAGE_PATH/MiniLoaderAll.bin
$IMAGE_PATH/update.img emmc_update.img -os_type:androidos -storage:emmc || pause
echo "Making emmc_update.img OK."

echo "regenernate $PACKAGE_FILE..."
if [ -f "$PACKAGE_FILE" ]; then
    rm -rf $PACKAGE_FILE
fi
```

```
./gen-package-file.sh $IMAGE_PATH_NOR > $PACKAGE_FILE

echo "start to make spinor update.img..."

./afptool -pack ./ $IMAGE_PATH_NOR/update.img $PACKAGE_FILE || pause
./rkImageMaker ${vendor_id_map[$PLATFORM]} $IMAGE_PATH_NOR/MiniLoaderAll.bin
$IMAGE_PATH_NOR/update.img spinor_update.img -os_type:androidos -storage:spinor || pause
echo "Making spi_update.img OK."

./rkImageMaker -merge ./update.img ./emmc_update.img ./spinor_update.img
rm ./spinor_update.img ./emmc_update.img

exit 0
```

Execute Script

RK3568:

```
./mkupdate.sh rk356x Image/ Image-nor/
```

RK3588:

```
./mkupdate.sh rk3588 Image/ Image-nor/
```

4.2 RK PCIe EP Dual Storage Solution Firmware Packaging

The packaging principle is the same as the RK Universal Dual Storage Solution.

Tool Directory

```
.
├── afptool
├── gen-package-file.sh
├── Image                       # Main storage firmware
│   ├── boot.img
│   ├── MiniLoaderAll.bin
│   ├── parameter.txt
│   └── userdata.img
├── Image-flash                 # Boot storage firmware
│   ├── MiniLoaderAll.bin
│   ├── normal_idb.img          # idblock.img, contains ddr.bin and spl.bin
│   ├── parameter.txt
│   ├── pcie_idb.img            # PCIe_idblock.img, the source code project bootloader-ep
defaults to output dual backup images, containing pcie.bin
│   └── uboot.img
├── mkupdate.sh
└── rkImageMaker
```

Instructions:

- For generating pcie_idb.img, please refer to the following commands and enter the rkbin directory:

```
# Generate loader
./tools/boot_merger RKBOOT/RK3588MINIALL_PCIE_EP.ini
$ls rk3588_pcie_loader_v1.00.bin
rk3588_pcie_loader_v1.00.bin

# Create idb, require spinand type alignment
./tools/programmer_image_tool -i rk3588_pcie_loader_v1.00.bin -b 128 -p 2 -t spinand -o ./

# Create dual backup pcie_idb.img
cat idblock.img > pcie_idb.img
cat idblock.img >> pcie_idb.img
rm idblock.img
```

- For generating normal_idb.img, please refer to the following commands and enter the rkbin directory:

```
# Generate loader
./tools/boot_merger RKBOOT/RK3588MINIALL.ini
$ls rk3588_spl_loader_v1.14.113.bin
rk3588_spl_loader_v1.14.113.bin

# Create idb, require spinand type alignment
./tools/programmer_image_tool -i rk3588_spl_loader_v1.14.113.bin -b 128 -p 2 -t spinand -o
./

# Create dual backup normal_idb.img
cat idblock.img > normal_idb.img
cat idblock.img >> normal_idb.img
rm idblock.img
```

Add Image-flash Directory and Set parameter Configuration

SPI Nor reference:

```
CMDLINE:mtdparts=rk29xxnand:0x00000080@0x00000000(gpt),0x00000400@0x00000080(pcie_idb),0x000
00b80@0x00000480(normal_idb),0x00000800@0x00001000(vnvm),0x00002000@0x00001800(uboot)
```

2KB page size SPI Nand reference:

```
CMDLINE:mtdparts=rk29xxnand:0x00000080@0x00000000(gpt),0x00000400@0x00000100(pcie_idb),0x000
00400@0x00000500(normal_idb),0x00000800@0x00001000(vnvm),0x00002000@0x00001800(uboot)
```

4KB page size SPI Nand (compatible with 2KB page size) reference:

```
CMDLINE:mtdparts=rk29xxnand:0x00000080@0x00000000(gpt),0x00000400@0x00000200(pcie_idb),0x000
00400@0x00000600(normal_idb),0x00000800@0x00001000(vnvm),0x00002000@0x00001800(uboot)
```

Modify mkupdate.sh Script to Support Dual Storage Packaging

Refer to the corresponding section of the "RK Universal Dual Storage Solution".

Execute the Script

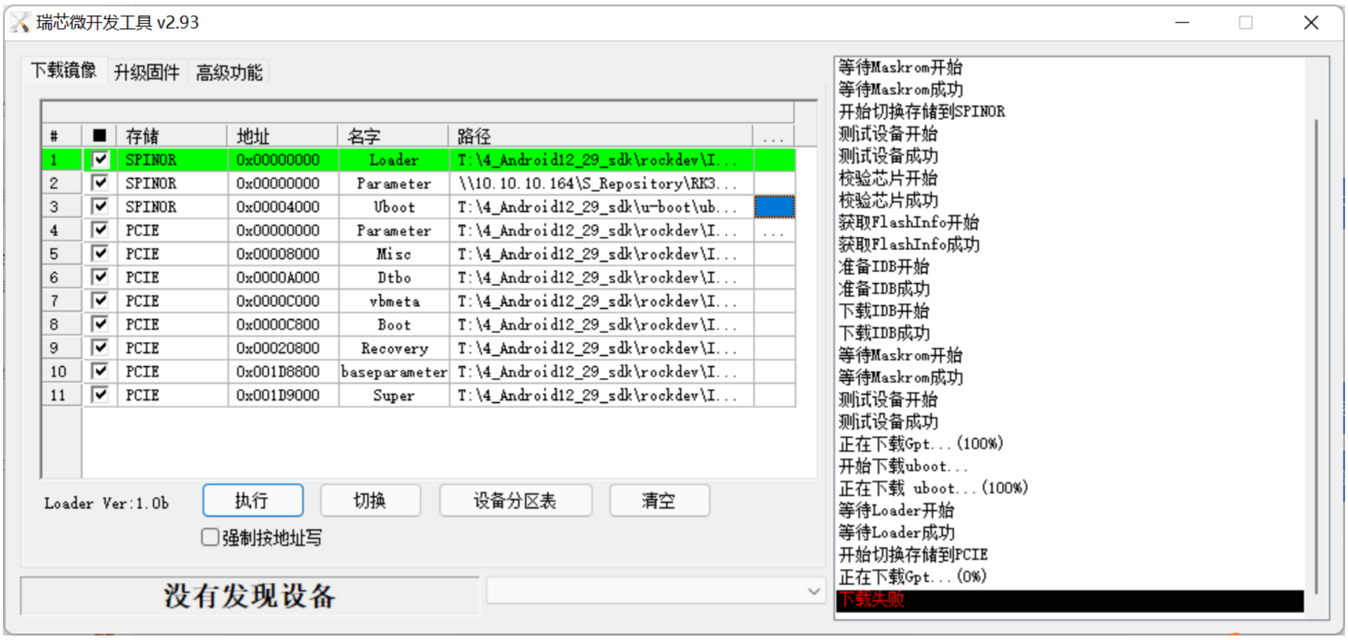
Refer to the corresponding section of the "RK Universal Dual Storage Solution".

5. Firmware Burning

The RK burning tool supports multi-storage burning methods.

Windows: Use the RKDevTool_Release_v2.93 or higher version tool, for detailed reference, read the "1.12 Multi-device Selection" section in the "Development Tools User Manual" included with the tool package.

One-time Burning



Burn Only SPI Nor

Enter the maskrom mode of the device and select only the SPINOR firmware for storage.

Burn Only NVMe

Enter the loader mode of the device and select only the PCIe firmware for storage.

6. OTA Upgrade

6.1 RK Universal Dual-Storage Scheme OTA Upgrade

Nor+ eMMC as an Example

- Nor Configuration:

  The Nor part includes the MiniLoader, parameter.txt, and uboot.img partitions, all of which can support

OTA upgrades. Here, we use the uboot partition as an example. **It is not recommended to upgrade the MiniLoader and parameter.txt partitions as the risk is high.**

The uboot partition is in the Nor storage device, while boot and rootfs are in the eMMC storage device. Therefore, it is necessary to add the partition information of uboot and the device configuration of Nor in the kernel.

Adding Partition Information:

Referring to the "RK Universal Dual-Storage Scheme Firmware Packaging" section, define the recovery dts mtdparts partition table based on the parameter.txt layout:

```
mtdparts=sfc_nor:0x00100000@0x00010000(idb),0x00100000@0x00200000(vnvm),0x00400000@0x0
0300000(uboot)
```

Adding Nor Device Configuration:

```
&sfc {
        status = "okay";
        pinctrl-names = "default";
        pinctrl-0 = <&fspim1_pins>;                 // Select actual IO
        flash@0 {
                compatible = "jedec,spi-nor";
                reg = <0>;
                spi-max-frequency = <75000000>;
                spi-rx-bus-width = <4>;
                spi-tx-bus-width = <1>;
        };
};
```

After adding successfully, check if the /dev/mtdblock device exists in the root file system. If the device does not exist, please check the configuration information and review the boot log for any spi-nor related errors, which may be due to incorrect iomux configuration or unsupported Nor model issues.

- eMMC Configuration:

  The partition information for eMMC is all in the parameter.txt of the eMMC device. As long as the corresponding partitions are added to the partition table, the root file system will generate the corresponding mmcblock devices.

Once the partition nodes for Nor and eMMC devices are generated, the RK OTA scheme can download the recovery patch through the following link and refer to the patch instructions for OTA upgrade. Custom upgrade schemes can freely choose methods to upgrade the partition nodes:

```
Link: [Download Recovery Patch](https://pan.baidu.com/s/1LiutCDJZ8BXiEN6X8U6teQ?pwd=nwm3)
Extraction Code: nwm3
```

6.2 RK PCIe EP Dual Storage Solution OTA Upgrade

The principle is the same as the RK universal dual storage solution OTA upgrade, with only the addition of partition information.

Based on the layout definition of parameter.txt in the "RK PCIe EP Dual Storage Solution Firmware Packaging" section, modify and add the recovery dts mtdparts partition table:

SPI Nor configuration:

```
mtdparts=sfc_nor:0x00080000@0x00010000(pcie_idb),0x00170000@0x00090000(normal_idb),0x0010000
0@0x00200000(vnvm),0x00400000@0x00300000(uboot)
```

2KB page size SPI Nand reference:

```
mtdparts=spi-
nand0:0x00000080@0x00000000(gpt),0x00000400@0x00000100(pcie_idb),0x00000400@0x00000500(norma
l_idb),0x00000800@0x00001000(vnvm),0x00002000@0x00001800(uboot)
```

4KB page size SPI Nand (compatible with 2KB page size) reference:

```
mtdparts=spi-
nand0:0x00000080@0x00000000(gpt),0x00000400@0x00000200(pcie_idb),0x00000400@0x00000600(norma
l_idb),0x00000800@0x00001000(vnvm),0x00002000@0x00001800(uboot)
```

7. SPI NOR VENDOR STORAGE Support

When using dual storage of SPI NOR + SSD (SATA or NVMe), data such as SN and MAC needs to be written to the SPI NOR.

SPI NOR requires defining a "vnvm" partition to store VENDOR STORAGE data, with both the starting position and size of the partition being multiples of 64KB.

7.1 Partition Table Modification

Add a "vnvm" partition in the SPI NOR partition table file "parameter.txt", with the unit being sector (512 bytes).
Reference:

```
CMDLINE: mtdparts=rk29xxnand:0x00000200@0x00000c00(vnvm),0x00003000@0x00000c00(uboot)
```

7.2 Adding SPI NOR Partition Table to Kernel

The Kernel requires the addition of an SPI NOR partition table within the dts CMDLINE, which should match the starting positions and sizes of the partitions defined in the "parameter.txt" file, with units in bytes. Reference:

```
CMDLINE: mtdparts=sfc_nor:0x00040000@0x00180000(vnvm),0x00600000@0x00200000(uboot)
```

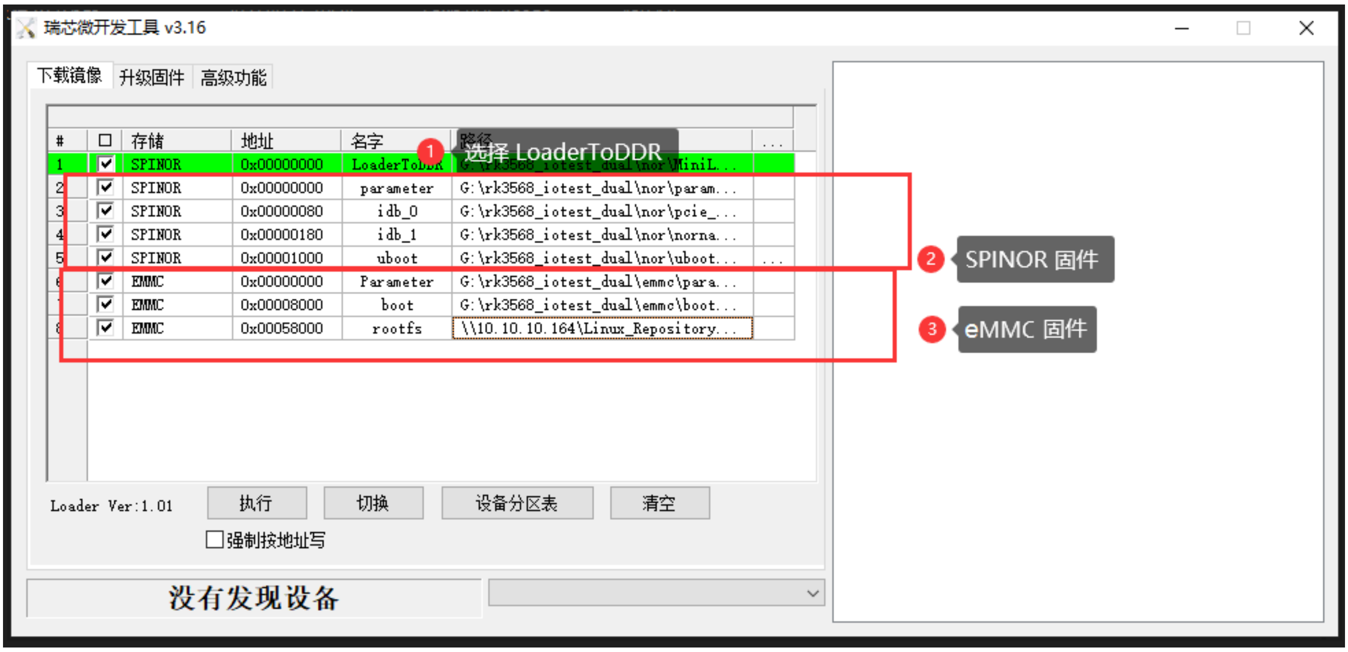
8. Common Issues

8.1 NOR Flash Firmware Partition Layout Trimming Suggestions

The default scheme in dual storage automatically arranges the idb image for the NOR flash upgrade Loader firmware, including the upgrade starting address and multiple backups. It is recommended to maintain the default scheme.

If you need to adjust the NOR flash firmware partition layout, the following suggestions are provided for the RK3568 NOR + eMMC scheme.

AndroidTool tool firmware partition layout:



AndroidTool tool configuration file modification:

```
MSC_VIDPID=
ADB_VIDPID=
MTP_VIDPID=
UVC_VIDPID=

#指定是否支持全速usb设备,当SUPPORTLOWUSB=TRUE时,增加全速usb设备支持,默认只支持高速usb设备
SUPPORTLOWUSB=true
#设置烧写固件时单次传输的数据带宽, 取值在[0-6]之间, 6代表1M,5代表512K, 0代表16K
FORCE_DATA_BAND=
MSC_TIMEOUT=30
ROCKUSB_TIMEOUT=30
#指定启动时加载的镜像配置文件,默认为config.cfg
DEFAULT_IMAGE_CONFIG=config.cfg
#下载Image镜像后是否进行设备重启
RESET_AFTER_DOWNLOAD=TRUE
#当设置FW_NOT_CHECK=TRUE时,固件加载时不进行完整性校验
FW_NOT_CHECK=TRUE
#当设置RB_CHECK_OFF=FALSE时,固件升级时才进行回读校验
RB_CHECK_OFF=
#LBA_PARITY=TRUE时,设备端开启写后校验
LBA_PARITY=
#NorFlash单个IDBlock
NOR_SINGLE_IDB=
#当设置CLOSE_CHECK_IDB=TRUE时,不检查IDBLOCK是否会被覆盖
CLOSE_CHECK_IDB=TRUE   4
#自动保存配置
AUTO=true
```

Explanation:

1. Change the Loader option to LoaderToDDR: Cancel the automatic layout of the idb image brought by the Loader option, and this Loader firmware is only used as a boot upgrade tool.
2. SPI NOR firmware: The parameter can be upgraded by the tool's automatic partition table in the form of a parameter item - parameter.txt file, and can also be upgraded in the form of a gpt item - gpt.img image.
3. eMMC firmware: The overall scheme is the same as the single storage eMMC scheme, and a parameter.txt file must be generated to correspond to the partition table.
4. Since the automatic layout of the idb image is canceled, the upgrade tool needs to turn off the idb image detection.

9. Appendix

9.1 RK3588s EVB1 Nor + NVMe Reference Patch

```
From 89825ff12bc92ba9dd9e063e9835ecb03b732b92 Mon Sep 17 00:00:00 2001
From: Jon Lin <jon.lin@rock-chips.com>
Date: Fri, 19 Nov 2021 20:57:52 +0800
Subject: [PATCH] TEST: uboot: rk3588s-tablet&evb1: nvme
```

```
Change-Id: I332fcceb8984712a4cd3eec053f813590f4e9bbe
Signed-off-by: Jon Lin <jon.lin@rock-chips.com>
---
 arch/arm/dts/rk3588-u-boot.dtsi | 34 +++++++++++++++++++++++++++++++++
 configs/rk3588_defconfig        | 12 ++++++++++++
 2 files changed, 46 insertions(+)

diff --git a/arch/arm/dts/rk3588-u-boot.dtsi b/arch/arm/dts/rk3588-u-boot.dtsi
index 3fe8054aac..25ebf26873 100644
--- a/arch/arm/dts/rk3588-u-boot.dtsi
+++ b/arch/arm/dts/rk3588-u-boot.dtsi
@@ -22,6 +22,33 @@
 		compatible = "rockchip,rk3588-secure-otp";
 		reg = <0x0 0xfe3a0000 0x0 0x4000>;
 	};
+
+	vcc12v_dcin: vcc12v-dcin {
+		u-boot,dm-pre-reloc;
+		compatible = "regulator-fixed";
+		regulator-name = "vcc12v_dcin";
+		regulator-always-on;
+		regulator-boot-on;
+		regulator-min-microvolt = <12000000>;
+		regulator-max-microvolt = <12000000>;
+	};
+
+	vcc3v3_pcie20: vcc3v3-pcie20 {
+		u-boot,dm-pre-reloc;
+		compatible = "regulator-fixed";
+		regulator-name = "vcc3v3_pcie20";
+		regulator-min-microvolt = <3300000>;
+		regulator-max-microvolt = <3300000>;
+		enable-active-high;
+		gpio = <&gpio4 RK_PB1 GPIO_ACTIVE_HIGH>;
+		startup-delay-us = <5000>;
+		vin-supply = <&vcc12v_dcin>;
+	};
+};
+
+&combphy2_psu {
+	u-boot,dm-pre-reloc;
+	status = "okay";
 };

 &firmware {
@@ -201,6 +228,13 @@
 	u-boot,dm-spl;
 };

+&pcie2x1l1 {
+	u-boot,dm-pre-reloc;
+	reset-gpios = <&gpio4 RK_PA2 GPIO_ACTIVE_HIGH>;
+	vpcie3v3-supply = <&vcc3v3_pcie20>;
```

```
+   status = "okay";
+};
+
 &pinctrl {
     u-boot,dm-spl;
     /delete-node/ sdmmc;
diff --git a/configs/rk3588_defconfig b/configs/rk3588_defconfig
index f65da00444..f9fbaa145b 100644
--- a/configs/rk3588_defconfig
+++ b/configs/rk3588_defconfig
@@ -226,3 +226,15 @@ CONFIG_RK_AVB_LIBAVB_USER=y
 CONFIG_OPTEE_CLIENT=y
 CONFIG_OPTEE_V2=y
 CONFIG_OPTEE_ALWAYS_USE_SECURITY_PARTITION=y
+CONFIG_CMD_PCI=y
+CONFIG_NVME=y
+CONFIG_PCI=y
+CONFIG_DM_PCI=y
+CONFIG_DM_PCI_COMPAT=y
+CONFIG_PCIE_DW_ROCKCHIP=y
+CONFIG_PHY_ROCKCHIP_NANENG_COMBOPHY=y
+CONFIG_PHY_ROCKCHIP_SNPS_PCIE3=y
+CONFIG_DM_REGULATOR_FIXED=y
+CONFIG_ROCKCHIP_BOOTDEV="nvme 0"
+CONFIG_EMBED_KERNEL_DTB_ALWAYS=y
+CONFIG_SPL_FIT_IMAGE_KB=2560
--
2.17.1
```

9.2 RK3568 EVB1 Nor + eMMC Reference Patch

```
diff --git a/configs/rk3568_defconfig b/configs/rk3568_defconfig
index d157307403..82801f3dcb 100644
--- a/configs/rk3568_defconfig
+++ b/configs/rk3568_defconfig
@@ -222,3 +222,5 @@ CONFIG_RK_AVB_LIBAVB_USER=y
 CONFIG_OPTEE_CLIENT=y
 CONFIG_OPTEE_V2=y
 CONFIG_OPTEE_ALWAYS_USE_SECURITY_PARTITION=y
+CONFIG_ROCKCHIP_EMMC_IOMUX=y
+CONFIG_ROCKCHIP_BOOTDEV="mmc 0"
```

Note:

- No need to enable Embedded Kernel DTB configuration.